Netflix Movie Recommendation Engine

Capstone Project Report

Machine Learning | Recommendation Systems | Streamlit Deployment

Student	Kummari Thirumala Raju
Project Domain	Data Science and Machine Learning
Dataset Size	5969 ratings, 100 users, 500 movies, 10 genres
Model Metrics	RMSE: 1.150, MAE: 0.972
Deliverables	EDA dashboard, recommendation engine, ML evaluation, Streamlit app, GitHub-ready source code

A recruiter-ready, product-style machine learning project for OTT movie personalization.

Table of Contents

1. Executive Summary
2. Problem Statement
3. Objectives
4. Dataset Description
5. Proposed System Architecture
6. Methodology
7. Exploratory Data Analysis
8. Recommendation Techniques
9. Model Evaluation
10. Streamlit Application Design
11. Project Folder Structure
12. Results and Business Insights
13. Skills and Resume Value
14. Future Enhancements
15. Conclusion

1. Executive Summary

This capstone project presents a Netflix-style movie recommendation engine that recommends movies using user rating behavior, movie genre, content similarity and popularity patterns. The project is designed as a complete data science product rather than a simple analysis notebook. It includes datacleaning, exploratory data analysis, recommendation logic, machine learning-based rating prediction, model evaluation and an interactive Streamlit web application.

The dataset used in this project contains 5969 user-movie rating records from 100 unique customers, 500 movies and 10 genres. The average rating is 3.44, with movie release years ranging from 1918 to 2005. The final application allows users to explore data trends, view popular movies, search similarmovies and generate personalized recommendations.

2. Problem Statement

OTT platforms such as Netflix, Amazon Prime and Disney+ Hotstar depend heavily on recommendation engines to keep users engaged. A large catalog creates choice overload: users may spend more time searching than watching. Recommendation systems solve this by predicting what a user is likely to enjoybased on ratings, interests and similar user behavior.

The capstone brief requires building a recommendation engine from the ground up, finding popular and liked genres, recommending suitable movies to users and identifying genres with best and worst ratings. This project addresses all these goals and extends them into a deployable Streamlitapplication.

3. Objectives

- Analyze rating behavior and identify common rating patterns.
- Find the most popular and most liked movie genres.
- Identify best-rated and worst-rated genres based on user ratings.
- Build popularity-based recommendations for new users.
- Build content-based recommendations using TF-IDF and cosine similarity.
- Build personalized user recommendations using user history and hybrid scoring.
- Train a machine learning model for rating prediction and evaluate it with RMSE and MAE.
- Deploy the complete workflow as an interactive Streamlit web application.

4. Dataset Description

Column	Description
CustomerID	Unique identifier for each customer/user
MovieID	Unique identifier for each movie
Rating	User rating from 1 to 5
Title	Movie title
Genre	Movie category
Year	Movie release year

The dataset does not contain missing values or duplicate rows after initial checks. The final dataset is suitable for both descriptive analytics and recommendation modeling.

5. Proposed System Architecture

The system follows a modular machine learning workflow. Raw data is loaded and cleaned, features are engineered, multiple recommendation models are built, trained objects are saved inside the models folder and the final app consumes these objects for fast inference.

- Data Layer: Netflix ratings CSV containing users, movies, ratings, genres and years.
- Processing Layer: Pandas cleaning, aggregation and feature engineering.
- Model Layer: popularity scoring, content similarity, hybrid scoring and rating prediction.
- Application Layer: Streamlit app with dashboard pages and user interaction.
- Deployment Layer: GitHub repository and Streamlit Cloud deployment.

6. Methodology

6.1 Data Cleaning

- Removed duplicate rows.
- Converted CustomerID, MovieID, Rating and Year to numeric format.
- Validated rating range from 1 to 5.
- Created movie-level statistics for average rating, rating count and popularity score.

6.2 Feature Engineering

- Created popularity_score = average_rating * log(1 + rating_count).
- Created combined text features using Title, Genre and Year for content-based filtering.
- Encoded CustomerID, MovieID and Genre using LabelEncoder for rating prediction.
- Created hybrid scores using movie rating, genre preference and popularity.

7. Exploratory Data Analysis

The EDA stage helps explain user behavior and gives business insights before building models. The most important finding is that Drama receives the highest number of ratings, showing that it dominates user activity in this dataset. Thriller is the best-rated genre, while Comedy has the lowestaverage rating.

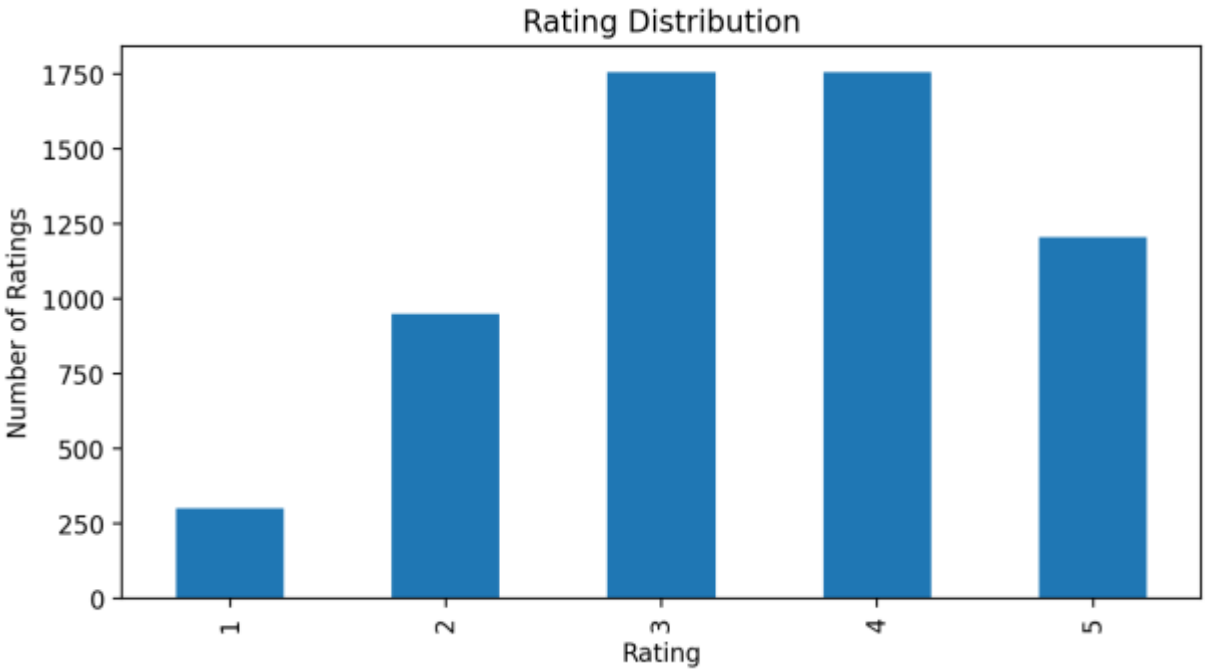

Figure 1: Distribution of ratings

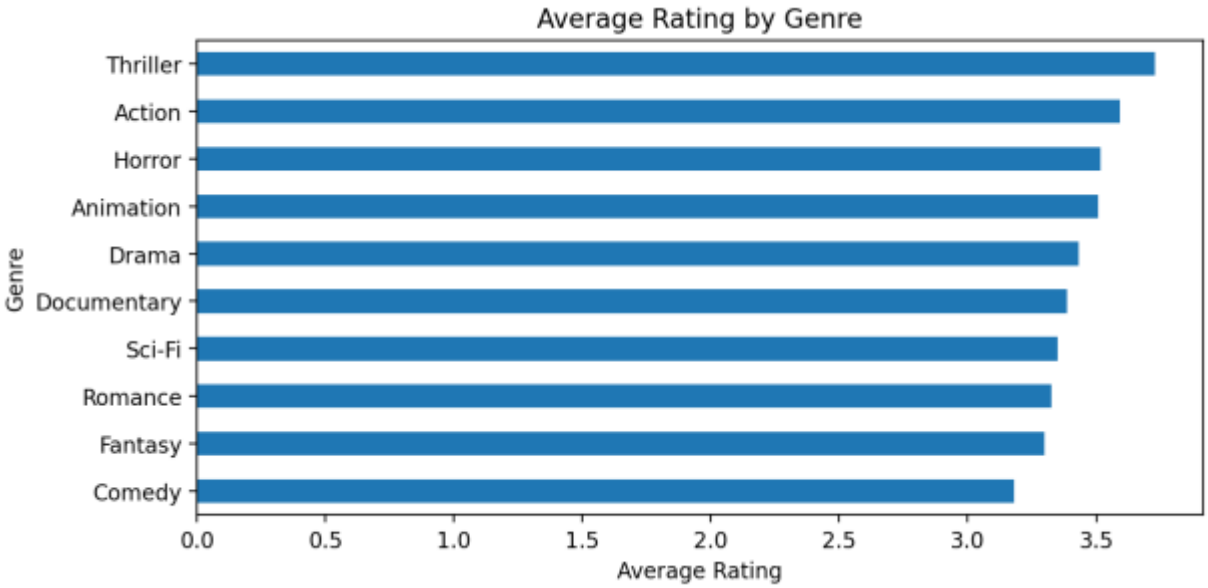

Figure 2: Average rating by genre

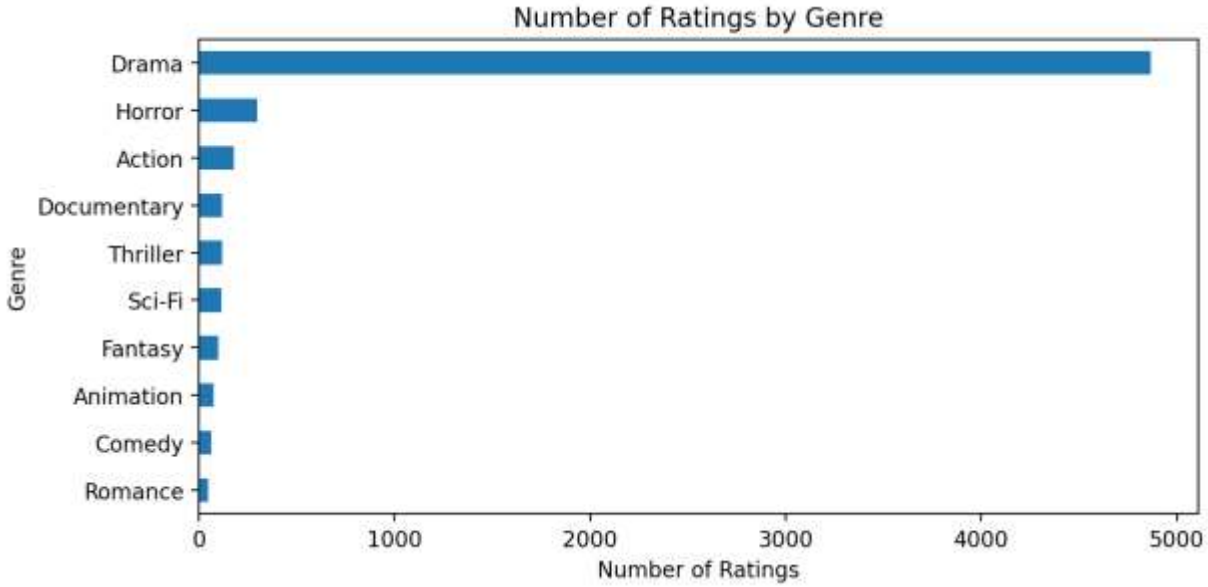

Figure 3: Number of ratings by genre

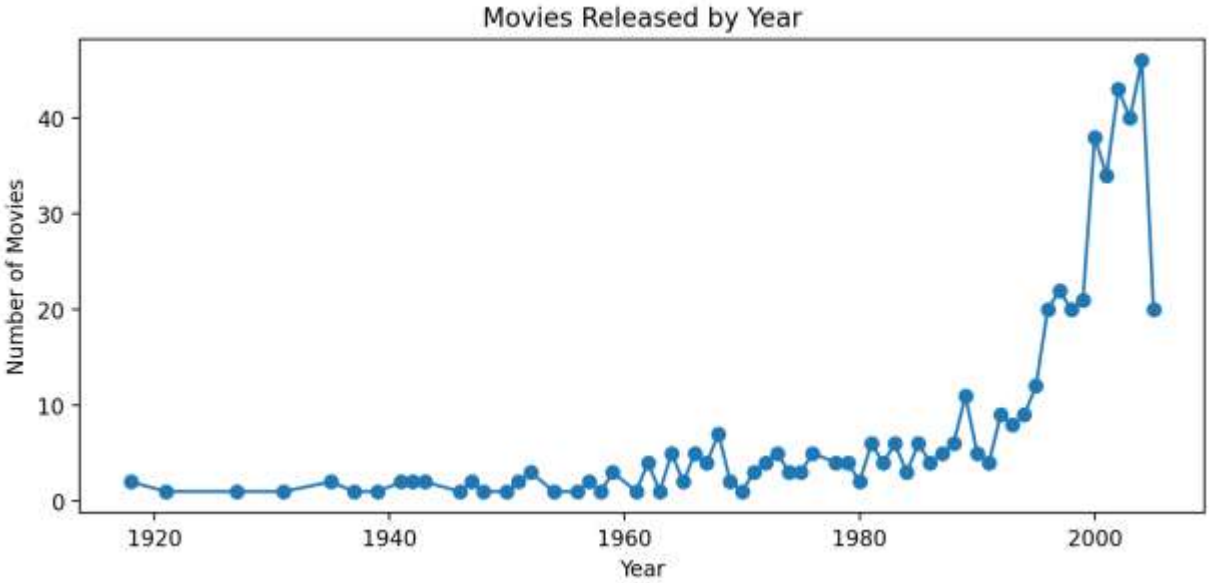

Figure 4: Movies released by year

Genre Summary

Genre	Average_Rating	Total_Ratings	Unique_Movies
Thriller	3.729	118	9
Action	3.593	177	15
Horror	3.517	296	25
Animation	3.507	73	6
Drama	3.431	4865	408
Documentary	3.387	119	9
Sci-Fi	3.351	114	10
Romance	3.327	49	4
Fantasy	3.299	97	8
Comedy	3.180	61	6

8. Recommendation Techniques

8.1 Popularity-Based Recommendation

This model recommends movies with strong average ratings and enough rating volume. It is useful for new users where no user history is available. The popularity score reduces the risk of recommending movies with high average ratings but very few ratings.

Formula: popularity_score = average_rating x log(1 + number_of_ratings).

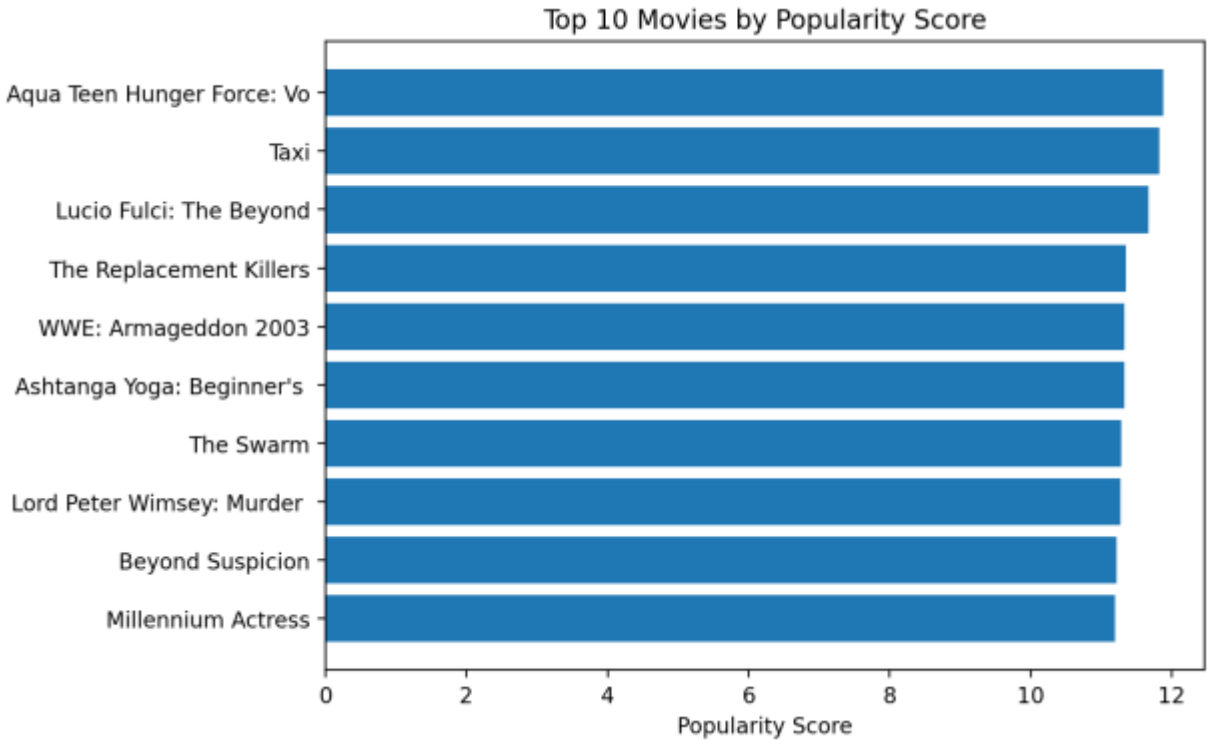

Figure 5: Top movies based on popularity score

8.2 Content-Based Recommendation

Content-based filtering recommends movies similar to a selected movie. The project combines title, genre and release year into text features, converts the text into vectors using TF-IDF and compares movies using cosine similarity. This powers the Movie Recommendation page in the Streamlit app.

8.3 User-Based Personalized Recommendation

User-based recommendation considers the movies a specific customer has already rated and calculates their preferred genres. Movies already watched by the user are removed from the recommendation pool. The system then ranks unseen movies using a hybrid score.

8.4 Hybrid Recommendation

The hybrid recommender combines average movie quality, the user genre preference and popularity. This improves recommendation quality because it does not depend on one single signal. The hybrid score used in the app gives higher importance to user preference and movie quality while still consideringpopularity.

9. Model Evaluation

A Random Forest Regressor was trained to predict movie ratings using encoded CustomerID, MovieID, Genre and Year. The model was evaluated with train-test split using RMSE and MAE. These metrics help measure how far predicted ratings are from actual ratings.

Model	RMSE	MAE	Use Case
Random Forest Regressor	1.150	0.972	Rating prediction

- RMSE penalizes large prediction errors more strongly.
- MAE gives the average absolute error in rating points.
- Lower values indicate better predictive performance.

10. Streamlit Application Design

The final product is an interactive Streamlit dashboard. It is designed for recruiters and evaluators to understand the project quickly without reading code first.

Page	Purpose
Home	Project overview and key dataset metrics
EDA Dashboard	Rating distribution, genre ratings, genre counts and year trends
Popular Movies	Genre filter and top-N popular movies
Movie Recommendation	Search/select a movie and get similar movies
User Recommendation	Enter customer ID, view history and get personalized recommendations
Model Evaluation	RMSE, MAE and rating prediction demo
Project Insights	Business interpretation and resume value

11. Project Folder Structure

```
netflix-recommendation-engine/
├── app.py
├── requirements.txt
├── README.md
├── data/
│   └── netflix_ratings.csv
├── models/
│   ├── rating_prediction_model.pkl
│   ├── user_encoder.pkl
│   ├── movie_encoder.pkl
│   ├── genre_encoder.pkl
│   ├── tfidf_vectorizer.pkl
│   ├── cosine_similarity.pkl
│   ├── movie_stats.pkl
│   ├── movies.pkl
│   └── model_metrics.pkl
├── src/
│   └── train_models.py
├── notebooks/
│   ├── 01_data_cleaning.ipynb
│   ├── 02_eda.ipynb
│   └── 03_model_building.ipynb
└── report/
    └── Netflix_Recommendation_Report.pdf
```

12. Results and Business Insights

- Best-rated genre: Thriller.
- Worst-rated genre: Comedy.
- Most-rated genre: Drama.
- Average rating across all users: 3.44.
- Drama dominates user activity, so it can be prioritized on the home page.
- Thriller has strong satisfaction and can be recommended to high-engagement users.
- Comedy has lower average rating and needs further analysis before promotion.

13. Skills and Resume Value

This project demonstrates a complete end-to-end machine learning workflow. It is suitable for Data Analyst, Data Science, Machine Learning and AI internship resumes because it includes both technical depth and business explanation.

- Python programming and Pandas data analysis.
- Exploratory data analysis with visualizations.
- Recommendation systems using popularity, content similarity and hybrid scoring.
- Machine learning model training and evaluation.
- Streamlit application development and deployment-ready project structure.
- GitHub documentation and professional project presentation.

14. Future Enhancements

- Add SVD matrix factorization or Neural Collaborative Filtering for stronger personalization.
- Use movie posters and metadata from external APIs.
- Add login-based user sessions and real-time feedback.
- Deploy the model with a REST API backend.
- Add A/B testing metrics such as click-through rate and watch completion rate.

15. Conclusion

The Netflix Movie Recommendation Engine successfully meets the capstone objectives by analyzing genre popularity, identifying best and worst-rated genres and recommending movies to users. The project goes beyond basic analysis by creating an interactive app with multiple recommendation techniquesand model evaluation. The final system is structured, deployment-ready and suitable for showcasing in a resume, GitHub portfolio and interview discussion.